Too Many Hands on the Throttle: A Pre-Registered Failure Study of Composed Reliability Agents

Abhishek Pandey
Meta
pandeyabhi1987@gmail.com

Abhinav Pandey
Independent Researcher
pandey.aby@gmail.com

Abstract—Production services run several autonomous reliability agents at once: an autoscaler, load shedders, concurrency and rate limiters, retry budgets, and outlier ejection. Each is designed, documented and validated alone, yet all act on one plant. We study how such agents fail together, and how to test for it. We distinguish three coupling channels. Actuation coupling sums gains past a delay margin no single agent can observe. Observation coupling lets one agent hide demand from another. Topological coupling redistributes load past a branching threshold. A model shows that observation coupling yields a threshold-ordering trap: when a shedder engages below the autoscaler's target, the fleet settles under-provisioned. An audit of Kubernetes, Envoy, Istio and Linkerd finds no configuration surface that coordinates provisioning with protection. In pre-registered tests on unmodified Kubernetes and Envoy, the trap reproduced in all 9 below-target trials across three shedder types and two application shapes (attainment 0.29–0.50, 2–3 of 5 replicas). All 8 above-target trials escaped (attainment $\geq$0.99). Our evidence for an arbitration remedy is negative or uninformative so far, and we report it as such. We release the simulator, testbed, pre-registrations and raw traces.

Index Terms—multi-agent interaction, testing autonomous systems, reliability control, autoscaling, load shedding, circuit breaking, control allocation, cascading failure, microservices

I. Introduction

Ask an operator how their service protects itself under load and you will get a list. There is an autoscaler that adds replicas when utilization rises. There is a load shedder that rejects low-priority work when latency climbs. There is outlier detection that ejects unhealthy endpoints. There is a concurrency limiter that caps in-flight requests, a rate limiter that caps outbound calls, a timeout policy, a retry budget, and a router that moves traffic away from trouble.

Each of these is a feedback controller. Each was designed in isolation, documented in isolation, and tuned in isolation. Most are configured through different surfaces, and in a large organization most are owned by different teams. And every one of them observes some function of the same underlying congestion state and acts, directly or indirectly, on that same state.

Control engineers have a name for this situation. When several actuators have authority over one plant, the assignment of corrective effort among them is the control allocation problem, and it has been treated rigorously for decades in aerospace, where an aircraft has more control surfaces than axes of motion [1], [2]. Service fleets have the same structure and do not use the same discipline. We allocate nothing. We run every controller at full authority and hope.

This paper argues that the hope is misplaced in a specific and predictable way, and that the resulting failure modes are invisible from inside any individual controller.

A. Reliability controllers as autonomous agents

Each of these mechanisms is, in the sense used by the agent literature, an autonomous agent: it perceives part of the environment through its own signal, decides on its own policy, and acts without a human in the loop or a protocol with its peers. A fleet is therefore an uncoordinated multi-agent system whose agents were each validated alone. We treat the problem as one of verifying and testing agent interaction: which properties of the composition can be checked statically from configuration, which require execution, and what an arbiter that mediates between agents buys. We do not claim these controllers learn or plan; the framing concerns autonomy and the absence of coordination, which is exactly what makes interaction failures hard to test for one agent at a time.

B. A failure no controller can see

Consider a fleet with an autoscaler and a load shedder, both configured with defaults that a careful operator would consider reasonable. Load rises. The shedder sees latency climb and begins rejecting requests, which is exactly its job, and latency recovers. The autoscaler, meanwhile, reads request rate at the service, sees it fall, concludes that demand has dropped, and scales the fleet down. Note that this requires an autoscaler configured on a request-rate (custom or external) metric; the Kubernetes default is CPU utilization [10], under which shed requests that never reach the pods have the same effect only if they would have consumed CPU. In our simulation (Section VII) the outcome is not the growing oscillation often described but a stable trap: when the shedder trips below the autoscaler's target, the fleet ratchets down and sheds persistently.

Every controller in this story behaved exactly as specified. The shedder shed under latency pressure. The autoscaler scaled to observed demand. Neither configuration is wrong. The fault is in the composition, and neither controller's telemetry contains enough information

to attribute it, because each one sees only its own loop closing correctly.

C. Contributions

1) A taxonomy and model of three coupling channels (Sections III–IV). The model includes the gain-superposition bound and the threshold-ordering trap, with the retunings that escape it.
2) A configuration audit (Section V) of what widely deployed agents sense and actuate, and which coordination hooks exist.
3) A pre-registered testing method (Section VI) for agent composition: static audit, model checks, and paired composition tests on unmodified components.
4) Reproduction of the trap on Kubernetes and Envoy (Section VIII): 9/9 below-target trials underprovisioned and 8/8 above-target trials escaped, across three shedders and two application shapes.
5) Honest negative evidence on arbitration (Section IX), including two failed hypotheses in simulation and a testbed budget that made matters worse.

We are explicit about scope. Our central evidence is the audit and pre-registered testbed experiments on the observation-coupling trap. Arbitration is discussed only briefly (Section IX), because its evidence so far is negative or uninformative. No claim here concerns production systems.

II. Background

A. The mechanisms

We use the following terms for the controllers a modern fleet runs. An autoscaler adjusts replica count from a utilization or demand signal. A load shedder or admission controller rejects a fraction of offered work, usually prioritized, when a congestion signal exceeds a threshold. A circuit breaker or outlier detector removes an endpoint from the serving set when its error rate or latency exceeds a threshold. A concurrency limiter caps in-flight requests. A rate limiter caps request rate. A timeout policy bounds per-call latency. A retry policy reissues failed calls, typically bounded by a retry budget. A router distributes traffic across endpoints and performs failover.

III. Three Coupling Channels

We distinguish coupling channels by what the controllers share.

A. Actuation coupling

Two controllers exhibit actuation coupling when their outputs move the same plant variable in the same direction.

A concurrency limiter and a rate limiter both bound offered load. A shedder and a rate limiter both bound admitted work. An autoscaler and an outlier detector both change effective serving capacity, in opposite directions. In each case the plant receives the sum of the corrections while each controller was tuned as though it were applying the whole correction alone.

Within a single proxy, Envoy already recognizes one such interaction: its retry budget is configured as a circuit-breaker threshold that bounds concurrent retries to a fraction of active requests [13].

B. Observation coupling

Two controllers exhibit observation coupling when one controller's actuation lies upstream of the other's measurement point, so that the second controller's input is a function of the first controller's output.

The autoscaler and shedder interaction of Section I is the canonical case. Shedding removes work before it reaches the point where request rate is measured, so the provisioning controller reads a demand signal that the protection controller is actively suppressing. The same structure appears whenever a protective mechanism is deployed at the edge of a measurement boundary, which is the normal place to deploy one.

We expect observation coupling to be the least visible of the three. Actuation coupling at least produces a plant that responds too strongly, which an operator may recognize as overshoot. Observation coupling produces a controller that is behaving correctly on false data.

C. Topological coupling

Two controllers exhibit topological coupling when one controller's action redistributes load and thereby changes the state the other controller observes at a different endpoint.

Ejection and failover routing are both redistributive. Removing an endpoint does not remove its traffic; it moves it. If survivors were already near saturation, the redistribution pushes them past the threshold that triggered the first ejection, and the process compounds. Retry amplification is a temporal variant: a timeout reduction converts slow successes into failures, failures into retries, and retries into additional offered load, so a tightening action increases the very quantity it was meant to reduce.

IV. A Model of Overlapping Authority

We model the fleet around an operating point and treat each coupling channel in turn. The models are deliberately minimal. Their purpose is to identify the structural condition under which composition fails, not to predict trajectories.

A. Setup

Let $x(t)$ denote a scalar congestion state at an endpoint, for instance queue depth or latency deviation from setpoint, normalized so that $x = 0$ is the operating point. Let controllers $i = 1, \ldots, m$ observe x and apply corrections u_i. Each controller has an actuation delay $\tau_i > 0$ comprising its measurement window, its decision interval, and the physical time for its action to take effect.

For an autoscaler τ includes container start time and is often tens of seconds; for a shedder it is milliseconds.

B. Actuation coupling breaks the delay margin

Consider m controllers with overlapping authority, each applying proportional feedback $u_i = -g_i x(t-\tau)$ with common delay τ and gain $g_i > 0$. Acting alone, controller i closes the loop

$$\dot{x}(t) = -g_i\, x(t-\tau) + d(t),$$

which is asymptotically stable if and only if $g_i\tau < \pi/2$, the classical delay margin for first-order feedback with pure delay. Define controller i's margin fraction

$$\rho_i \;=\; \frac{g_i\tau}{\pi/2} \;\in\; (0,1).$$

A controller tuned to consume half its delay margin has $\rho_i = 1/2$.

Proposition 1 (Gain superposition). If the m controllers act concurrently on the same plant variable, the composed loop is

$$\dot{x}(t) = -\Big(\textstyle\sum_{i=1}^{m} g_i\Big)\, x(t-\tau) + d(t),$$

which is asymptotically stable if and only if $\sum_{i=1}^{m} \rho_i < 1$: marginally stable at equality and unstable above it. In particular, any two controllers each tuned to consume half or more of the delay margin compose to a loop that is not asymptotically stable, despite each being stable in isolation.

Proof sketch: Superposition of the corrections gives the stated composed loop, whose stability condition is $(\sum_i g_i)\,\tau < \pi/2$. Dividing by $\pi/2$ yields $\sum_i \rho_i < 1$. ∎

Corollary 1 (Local tuning cannot prevent this). The margin fraction ρ_i is determined by controller i's own configuration and the plant. The stability condition depends on $\sum_j \rho_j$. No configuration surface enumerated in Section V exposes ρ_j for $j \neq i$, nor any proxy for it (Table I). The condition is therefore not evaluable, let alone enforceable, at any individual controller.

Corollary 1 is the practical heart of the actuation-coupling argument. The failure is not that operators tune badly. It is that the quantity governing stability is not exposed anywhere an operator could read it. Adding a second well-tuned controller to a well-tuned system can be unsafe, and nothing in the audited tooling says so.

Unequal delays complicate the algebra without changing the conclusion. When τ_i differ, the composed characteristic equation $s + \sum_i g_i e^{-s\tau_i} = 0$ replaces the single-delay form. Numerically, with ρ_i defined on each controller's own delay, $\sum_i \rho_i < 1$ remains sufficient but becomes conservative: instability onset occurred at $\sum \rho = 1.20$, 1.79, and 2.00 for equal-ρ pairs with delay ratios 1:2, 1:10, and 1:100 (Section VII). Widely separated delays, such as shedder and autoscaler, are therefore the least prone to this channel; it bites hardest between mechanisms of similar speed.

C. Observation coupling creates a threshold-ordering trap

Now consider two controllers with different roles. A fast controller F reduces offered load with output $u_F \geq 0$, and a slow controller S provisions capacity from a measured demand signal. Crucially, F acts upstream of S's measurement point, so S observes

$$\hat{\lambda} \;=\; \lambda - u_F$$

rather than the true offered load λ. Let capacity be $\mu = \mu_0 + u_S$ with $u_S = g_S(\hat{\lambda} - \mu)$, let $u_F = g_F\, x$ with $g_F > 0$, and let congestion respond to capacity with sensitivity $\kappa = -\partial x/\partial\mu > 0$.

Proposition 2 (Signal masking). The path $x \to u_F \to \hat{\lambda} \to u_S \to \mu \to x$ has positive sign. Including the direct path by which u_F itself relieves congestion, the linearization $\dot{x} = -g_F x - \delta\mu$, $\dot{\delta\mu} = g_S(-g_F x - \delta\mu)$ has eigenvalues 0 and $-(g_F + g_S)$: the masked operating point is neutrally stable, not unstable. Capacity is not restored toward true demand; any shed level consistent with $\delta\mu = \delta\lambda - g_F x$ is an equilibrium.

Proof sketch: The system matrix $\begin{pmatrix} -g_F & -1 \\ -g_S g_F & -g_S \end{pmatrix}$ has determinant 0 and trace $-(g_F + g_S)$. ∎

Corollary 2 (Threshold-ordering trap). If the load reducer trips at a utilization u_F^* below the provisioner's target u_S^*, admitted load is held near $u_F^*\mu < u_S^*\mu$, so the provisioner reads demand below its target and never scales up (and may scale down). Changing g_F, g_S, or smoothing windows does not alter this fixed point. It is escaped either structurally, by severing the dependence of $\hat{\lambda}$ on u_F (P2), or by retuning the thresholds so that $u_S^* < u_F^*$.

In simulation (Section VII) a shedder tripping at 60% utilization under a 70% autoscaler target held the fleet at 6–8 replicas where 18 were needed, shedding 64–70% of load, across every combination of shedder gain and stabilization window we tried; lowering the autoscaler target to 55% removed the trap entirely. The trap is thus a configuration-ordering hazard that no single controller's surface flags, but it is not immune to retuning.

D. Topological coupling has a branching threshold

Let an endpoint be ejected and its load redistributed over the surviving members of its pool. Let Γ denote the expected number of additional endpoints driven past the ejection threshold by the redistribution from one ejection, given the current load margin distribution across the pool.

Proposition 3 (Cascade subcriticality). Treating ejections as a branching process with mean offspring Γ, the ejection cascade terminates almost surely if and only if $\Gamma < 1$. When $\Gamma > 1$ the cascade reaches the pool's live-endpoint floor with positive probability.

The practical content is that Γ is computable before acting, from the topology and the current per-endpoint headroom, which is what makes topological coupling the most tractable of the three channels. Existing guards (Envoy's max_ejection_percent and panic threshold [11], [12], Istio's minHealthPercent [14]) bound ejection by fixed fractions rather than by Γ.

V. Does the Premise Hold in Practice?

The model predicts trouble under two conditions: overlapping authority, and independent configuration with no shared signal. This section tests whether both conditions hold in widely deployed software. We use three kinds of public evidence and deliberately avoid proprietary incident data, so that every claim here is checkable by a reader.

A. Configuration surface audit

Method: for each mechanism, read the upstream documentation and configuration reference, and record four fields. The signal the mechanism reads. The plant variable it actuates. The tuning parameters it exposes. Whether any documented hook allows it to observe or defer to a peer mechanism. Table I gives the schema and the mechanisms in scope. We audit Kubernetes horizontal pod autoscaling and cluster autoscaling, Envoy outlier detection, circuit breakers, adaptive concurrency, local and global rate limiting, and retry budgets, Istio destination-rule outlier detection, and Linkerd retry budgets.

Findings. (i) Several groups actuate one variable: three mechanisms move the serving set, three bound admitted load, two add retries, and the autoscaler and ejection move capacity in opposite directions. (ii) We found no documented hook linking autoscaling to any protection mechanism. (iii) Contrary to our prior expectation of a uniformly empty column, there are narrow intra-proxy hooks: Envoy's retry budget is a circuit-breaker threshold [13]; Envoy re-admits ejected hosts in panic mode and caps ejections [11], [12]; Istio disables outlier detection when healthy hosts fall below minHealthPercent [14]. (iv) Envoy's adaptive concurrency rejects with 503, a "gateway failure" that can feed consecutive-gateway-failure ejection [11], an observation-coupling path inside one proxy. Our claim is therefore narrowed to: no audited surface coordinates provisioning with protection.

B. Incident catalogue

Public postmortems can show that these interactions occur outside simulation. We use them for existence and mechanism, not for prevalence. This is a deliberate methodological limit. Vendors publish selectively, and most writeups lack the detail needed to diagnose a controller interaction, so any rate computed over published incidents would be an artifact of disclosure policy rather than a property of systems.

Method: rather than inherit a third-party curated list, whose maintenance we cannot depend on, we index primary sources directly. We search vendor incident archives from major cloud and infrastructure providers, apply stated inclusion criteria, and code each admitted incident against the three channels of Section III. Both authors will code independently and report agreement. This catalogue is not yet compiled. The resulting index, with inclusion criteria and per incident coding, is released as an artifact so that the catalogue does not inherit our own maintenance either.

C. Practitioner literature

Independent of any dataset, the interactions we model are already named in the canonical operational literature. The Google SRE treatment of cascading failures describes overload propagation and the role of retries in amplifying it [6]. The Amazon Builders' Library treats retry amplification and load shedding as system-level concerns rather than local ones [7], [8]. We cite this literature as evidence that the phenomena are recognized, and position our contribution as supplying the model and the remedy that it does not.

VI. Testing Method

We test agent composition in three layers, each preregistered in a public PREREG.md before the corresponding runs (v1–v3b). (i) A static audit (Section V) of what each agent senses and actuates, to find pairs that share a variable or a signal. (ii) Model checks in a deterministic simulator, to see whether the structural conditions of Section IV behave as stated. (iii) Composition tests on unmodified components: two agents, each in a documented configuration, under open-loop load, with a pass/fail threshold fixed in advance and a paired arm that reverses one ordering. A trial is valid only if its load phase completes and its metric sampling has no gap above 10 s. Invalid trials are rerun and reported.

VII. Model Checks in Simulation

Deterministic simulator; 20–30 seeds; 95% normal-approximation CIs. Actuation coupling. With $m = 1..5$ controllers each at $\rho = 0.6$ on a common delay, independent composition diverges for every $m \geq 2$, as Proposition 1 predicts. With unequal delays, instability onset moved to $\sum \rho = 1.20$, 1.79 and 2.00 for delay ratios 1:2, 1:10 and 1:100. Observation coupling. A shedder tripping at 60% under a 70% autoscaler target held a fluid fleet at 6–8 replicas of 18 needed, across all 12 combinations of gain and stabilization window. Lowering the target to 55% escaped. Topological coupling. In a 50-host pool, ungated ejection collapsed the pool at mean utilization ≥ 0.9 and never below; predicted $\Gamma \geq 1$ matched realized collapse in 180/180 runs, but outcomes were bimodal.

VIII. Testing the Threshold-Ordering Trap

Setup. Single-node k3s v1.31.4 on one 8-vCPU VM. A Python service has a capacity of about 50 req/s per pod (4 workers, 80 ms). Each pod has an Envoy v1.31.4 sidecar

TABLE I
Configuration audit (Envoy v1.39.3, Istio 1.31, Linkerd edge-26.10.1, Kubernetes docs as of 2026-10-11). Per-cell source URLs in audit/mechanisms.csv. †Cluster autoscaler row not re-verified.

Mechanism	Reads	Actuates	Coordination hook
K8s HPA	CPU/mem (default); custom/external metrics	replicas	none (accepts any metric)
K8s cluster autoscaler†	pending pods	nodes	none
Envoy outlier detection	5xx, gateway, local-origin errors	serving set	partial: max_ejection_percent; panic threshold
Envoy circuit breakers	in-flight counts	admitted load	hosts retry budget
Envoy adaptive concurrency	latency vs. minRTT	concurrency	none
Envoy local rate limit	request count	admitted load	none
Envoy retry budget	active+pending requests	retries	yes: bounded to % of active requests
Istio outlier detection	5xx, gateway errors	serving set	yes: minHealthPercent gate
Linkerd retries	response status	retries	none (legacy retryRatio)
Linkerd circuit breaking	failure accrual	serving set	none

TABLE II
Trap experiments, per arm (original validity rule). 5 replicas are needed.

Shedder	Arm	n	Attain.	Final repl.
Token bucket	below	3	0.375 (all)	2 (all)
	above	3	1.000 (all)	5 (all)
Adaptive conc.	below	3	0.285–0.286	2 (all)
	above	3	1.000 (all)	5 (all)
Two-tier	below	3	0.500 (all)	3 (all)
	above	2	0.992–0.999	5 (all)

shedder. An edge Envoy round-robins over pods. An autoscaling/v2 HPA scales on per-pod 2xx rate (Prometheus with prometheus-adapter), with a target of 35 req/s (0.7 of capacity), 2–8 replicas, and default stabilization. Open-loop load (vegeta): 50 req/s for 60 s, then 160 req/s for 480 s, which needs 5 replicas. Metrics cover the final 240 s. Each experiment pairs a below-target arm (shedding engages below the HPA target) with an above-target arm.

- Token bucket (PREREG v2): local_ratelimit at $R = 30$ vs. 40 req/s per pod.
- Adaptive concurrency (v3b): adaptive_concurrency pinned to limit $L = 2$ vs. 6, rejecting with 503.
- Two tiers (v3b): sidecar at 40 req/s plus an edge local_ratelimit at 80 req/s total vs. edge off.

Result. All pre-registered verdicts PASS (T1–3, AC1–3, TT1–3). In all 9 below-target trials the fleet ended under-provisioned (2–3 of 5 replicas, attainment $\leq$0.50). All 8 above-target trials reached 5 replicas with attainment $\geq$0.99. In the token-bucket case the HPA read 30 req/s per pod against a 35 req/s target and never scaled. Each controller behaved as documented; the failure lives only in the composition.

Validity notes. One above-target two-tier trial failed the gap rule. A relaxed rule added mid-campaign (gap $\leq$600 s after kubectl stalls) admits it and changes no verdict; the original rule is primary. One RQ1 trial truncated by a VM rollback was rerun. The pinned adaptive-concurrency limit tests the 503 path, not the gradient controller's dynamics. The box kernel lacks the iptables statistic match, so the edge Envoy, not kube-proxy, balances load.

IX. Toward Arbitration

The model suggests an arbiter, Concerto, that turns controllers into advisors. It has one principle per channel. P1: one severity estimate and a bounded authority budget allocated across actuators, so the summed margin fraction is bounded by construction [1]. P2: serve each controller a counterfactual input, e.g. admitted plus shed load for the autoscaler. P3: admit a redistributive action only if projected $\Gamma < 1$. P4: route slow disturbance bands to slow actuators. Each is implementable from outside the controllers. The evidence for it is negative or uninformative:

- Simulation, full arbiter (PREREG v1). The arbiter reached attainment 0.916 [0.914, 0.917] against 0.993 for a fixed expert retuning (lower HPA target, smaller gains). It did this at 0.69× the retuning's replica cost, but failed RQ3. Retuning also escaped the trap, failing RQ2. P1 alone did stabilize superposed gains (5.77 s settling for every m).
- Testbed, P1 stand-in (v3b). A static budget of edge 50 plus sidecar 25 req/s served 0.250 of load in 3/3 trials, worse than independent two-tier shedding (0.500). Its per-pod share (25) sat below the HPA target (35), so it recreated the trap. It lowered rates rather than allocating by severity, so it is not P1 as designed.
- Testbed, P3 (v3b). One failing endpoint among 10 produced about 0.3% errors and no cascade. All four ejection policies scored 0.997, so the test could not discriminate.

The lesson for design is that a budget must respect the threshold ordering it protects. A threshold-aware P1 and a pool that actually cascades are pre-registered as PREREG v4. Their results will be reported whatever they are.

X. Related Work

Control allocation and feedback computing. Assigning corrective effort among redundant actuators is well developed in aerospace [1], [2], and feedback control of

computing systems is treated systematically by Hellerstein et al. [15]. We borrow the allocation view and add two channels the aerospace formulation lacks: actuators that corrupt each other's sensors, and actions that redistribute load across a graph.

Autoscaling. Autoscaling is studied as a control and prediction problem [3]; production systems such as Autopilot [16] and FIRM [17] tune resources from observed signals. These works assume the autoscaler's input reflects demand. Corollary 2 and our testbed show when a co-located shedder breaks that assumption.

Overload and admission control. Server overload control has a long lineage [4]. DAGOR coordinates load shedding across WeChat's microservice graph [18], and Breakwater uses credits to admit RPCs at microsecond scale [19]. Adaptive concurrency limits import congestion control into the application layer [5]. These systems coordinate one mechanism across services. We study uncoordinated different mechanisms acting on one service.

Metastable failures and retry storms. Bronson et al. define metastable failures, in which a trigger pushes a system into a bad state sustained by its own feedback (often retries) [20]. Huang et al. find such failures across production incidents [21]. Retry amplification is a canonical practitioner concern [6], [7]. Our threshold-ordering trap is related but distinct. It is a stable equilibrium of two individually correct controllers and needs no trigger beyond a load step. It persists because each agent's own loop closes correctly.

Cascading failure. The branching threshold of Proposition 3 is standard in cascade models. Our use is as an admission gate on an agent's action; Envoy's ejection cap and Istio's minHealthPercent are fixed-fraction precursors [11], [14].

Testing agent interaction. Agents validated alone can fail together. Our contribution to this venue is a test method of three parts: a static audit of what each agent senses and actuates, pre-registered composition tests on real components, and an arbiter evaluated against an honest baseline.

XI. Threats to Validity

Construct validity. Attainment (2xx over offered) and replica count are proxies for user-facing objectives. Envoy's local_ratelimit per pod stands in for a capacity-relative shedder. The adaptive-concurrency variant is part of the pending campaign.

Internal validity. Thresholds were pre-registered (PREREG.md) before each run. Trials with sampling gaps above 10 s or truncated load phases are discarded and rerun, and we report the one such rerun. The box hosting the testbed paused and was once rolled back to a snapshot; the gap check exists for this reason. The Γ controller includes a disclosed exception that allows ejecting a host with under 10% success.

External validity. One node, synthetic services, and two or three mechanisms at a time. Production fleets are larger, heterogeneous, and configured differently. The simulator covers scale only in a model sense.

Conclusion validity. Testbed trials number two or three per arm. The effects are large and consistent (attainment 0.375 vs. 1.000 in every trial), but small n precludes fine-grained estimates. Simulator CIs are normal approximations over 20–30 seeds.

Negative results. The full arbiter lost to expert retuning in simulation (RQ2/RQ3). We report this rather than tune the baseline down.

XII. Artifact Availability

The simulator, audit table with per-cell source URLs, testbed manifests, trial scripts, raw replica traces, analysis code, and the pre-registrations (PREREG.md v1–v3, committed before the corresponding runs) are public at https://github.com/pandeyaby/CONCERTO. The testbed is reproducible on a single VM at no cost (testbed/setup.sh).

XIII. Conclusion

Reliability agents that are each correct can fail together, and the failures can be predicted, audited, and reproduced. The threshold-ordering trap reproduced on unmodified Kubernetes and Envoy in every pre-registered below-target trial, across three shedders and two application shapes, and reversing the ordering escaped it every time. No audited configuration surface would have warned the operator. Remedies are harder. A plain retuning beat our simulated arbiter, and a naive shared budget recreated the trap it was meant to prevent. We therefore offer a tested account of failure and a method for finding such failures, with arbitration as pre-registered future work.

References

[1] W. C. Durham, "Constrained control allocation," Journal of Guidance, Control, and Dynamics, vol. 16, no. 4, pp. 717–725, 1993.

[2] T. A. Johansen and T. I. Fossen, "Control allocation: A survey," Automatica, vol. 49, no. 5, pp. 1087–1103, 2013.

[3] T. Lorido-Botrán, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," Journal of Grid Computing, vol. 12, no. 4, pp. 559–592, 2014.

[4] M. Welsh and D. Culler, "Adaptive overload control for busy internet servers," in USENIX Symposium on Internet Technologies and Systems (USITS), 2003.

[5] Netflix Technology Blog, "Performance under load: Adaptive concurrency limits," 2018. [Online]. Available: https://netflixtechblog.medium.com/performance-under-load-3e6fa9a60581

[6] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, 2016, ch. 22, "Addressing Cascading Failures."

[7] M. Brooker, "Timeouts, retries, and backoff with jitter," Amazon Builders' Library. [Online]. Available: https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

[8] D. Yanacek, "Using load shedding to avoid overload," Amazon Builders' Library. [Online]. Available: https://aws.amazon.com/builders-library/using-load-shedding-to-avoid-overload/

[9] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in ASPLOS, 2019.
[10] Kubernetes, "Horizontal pod autoscaling." [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/, accessed Oct. 2026.
[11] Envoy v1.39.3, "Outlier detection." [Online]. Available: https://www.envoyproxy.io/docs/envoy/v1.39.3/intro/arch_overview/upstream/outlier
[12] Envoy v1.39.3, "Panic threshold." [Online]. Available: https://www.envoyproxy.io/docs/envoy/v1.39.3/intro/arch_overview/upstream/load_balancing/panic_threshold
[13] Envoy v1.39.3, "Circuit breakers (config.cluster.v3)." [Online]. Available: https://www.envoyproxy.io/docs/envoy/v1.39.3/api-v3/config/cluster/v3/circuit_breaker.proto
[14] Istio 1.31, "Destination rule." [Online]. Available: https://istio.io/v1.31/docs/reference/config/networking/destination-rule/
[15] J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury, Feedback Control of Computing Systems. Wiley-IEEE Press, 2004, doi:10.1002/047166880X.
[16] K. Rzadca et al., "Autopilot: Workload autoscaling at Google," in Proc. EuroSys, 2020, doi:10.1145/3342195.3387524.
[17] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in Proc. USENIX OSDI, 2020.
[18] H. Zhou et al., "Overload control for scaling WeChat microservices," in Proc. ACM SoCC, 2018, doi:10.1145/3267809.3267823.
[19] I. Cho, A. Saeed, J. Fried, S. J. Park, M. Alizadeh, and A. Belay, "Overload control for μs-scale RPCs with Breakwater," in Proc. USENIX OSDI, 2020.
[20] N. Bronson, A. Aghayev, A. Charapko, and T. Zhu, "Metastable failures in distributed systems," in Proc. HotOS, 2021, doi:10.1145/3458336.3465286.
[21] L. Huang et al., "Metastable failures in the wild," in Proc. USENIX OSDI, 2022.